DOCANVIL

CONTENTS

Home

GUIDES

Configuration

Getting Started

Localisation

Migration

PDF Export

Theming

WRITING CONTENT

Components

Front Matter

Markdown

REFERENCE

CSS Variables

Project Structure

Development

Roadmap

DocAnvil

A static documentation generator that turns Markdown into beautiful HTML sites.

Quick Start

```
# Install
cargo install --path .

# Create a new docs project
docanvil new my-docs

# Start the dev server with live reload
cd my-docs
docanvil serve
```

Open http://localhost:3000 in your browser and start writing.

Features

- **Markdown with GFM** — tables, task lists, strikethrough, footnotes, and front matter via comrak
- **Wiki-links** — connect pages with double-bracket links
- **Components** — notes, warnings, tabs, code groups, and mermaid diagrams using `::: directive` blocks
- **Full-text search** — client-side search powered by MiniSearch.js with a build-time JSON index
- **Mermaid diagrams** — flowcharts, sequence diagrams, and more via `::: mermaid` blocks
- **Theming** — CSS variables, custom stylesheets, and full template overrides with Tera
- **Localisation** — multi-language docs with per-locale URLs, navigation, search, and a language switcher
- **Live reload** — edit a file and your browser refreshes automatically
- **Static output** — build to plain HTML and deploy anywhere

Explore the Docs

SECTION	WHAT YOU'LL LEARN
Installation	Install DocAnvil and create your first project
Configuration	Customize `docanvil.toml` and `nav.toml`
Theming	CSS variables, custom CSS, and template overrides
Localisation	Multi-language docs with locale suffixes and language switcher
Markdown	All supported Markdown and GFM features
Links & Popovers	Wiki-link syntax and inline popovers
Components	Notes, warnings, tabs, and code groups
CLI Commands	Every command, flag, and option
Project Structure	Directory layout and page discovery
CSS Variables	Complete variable reference with defaults

> Getting started?
>
> Run `docanvil serve` in your project directory and open your browser — every change you save will appear instantly.

Configuration

DocAnvil uses two configuration files at the root of your project: `docanvil.toml` for project settings and `nav.toml` for navigation structure.

docanvil.toml

Minimal Full

```
[project]
name = "My Docs"
```

```toml
[project]
name = "My Docs"
content_dir = "docs"
[build]
output_dir = "dist"
base_url = "/my-project/"

[theme]
custom_css = "theme/custom.css"
color_mode = "both"

[theme.variables]
color-primary = "#059669"
font-body = "Georgia, serif"

[search]
enabled = true

[charts]
enabled = true
mermaid_version = "11"

[locale]
default = "en"
enabled = ["en", "fr"]
auto_detect = true

[locale.display_names]
en = "English"
fr = "Français"

[locale.flags]
en = "🇺🇸"

[pdf]
cover_page = true
author = "Your Name"
paper_size = "A4"
```

Required field

The `name` field under `[project]` is required. DocAnvil will fail to load without it.

`[project]` Section

KEY	DEFAULT	DESCRIPTION
`name`	*(required)*	Project name displayed in the sidebar and page titles
`content_dir`	`"docs"`	Directory containing your Markdown files

`[build]` Section

KEY	DEFAULT	DESCRIPTION
`output_dir`	`"dist"`	Directory where the static site is generated
`base_url`	`"/"`	URL path prefix for subfolder deployments (e.g. `"/my-project/"`)
`site_url`	`None`	Full site URL (e.g. `"https://example.com/"`) for canonical URLs, hreflang tags, and sitemap

> Recommended for i18n
>
> Setting `site_url` is strongly recommended when using localisation. It enables absolute hreflang URLs, canonical `<link>` tags, and `og:url` meta tags — all important for multilingual SEO.

`[theme]` Section

KEY	DEFAULT	DESCRIPTION
`name`	`None`	Reserved for future theme selection
`custom_css`	`None`	Path to a custom CSS file loaded after the default theme
`color_mode`	`"light"`	Color mode: `"light"`, `"dark"`, or `"both"` (light + dark with toggle)
`variables`	`{}`	CSS variable overrides injected as `:root` properties

Variables are specified as key-value pairs where the key is the CSS variable name (without `--`) and the value is any valid CSS value:

```
[theme.variables]
color-primary = "#059669"
color-bg = "#fafafa"
font-body = "Inter, sans-serif"
content-max-width = "960px"
```

See CSS Variables for the complete list of available variables.

`[search]` Section

KEY	DEFAULT	DESCRIPTION
`enabled`	`true`	Enable or disable full-text search

When enabled, DocAnvil generates a `search-index.json` file at build time and adds a search input to the header. Search is powered by MiniSearch.js, loaded from a CDN on first use. Set `enabled = false` to remove the search UI and skip index generation.

`[charts]` Section

KEY	DEFAULT	DESCRIPTION
`enabled`	`true`	Enable or disable Mermaid diagram rendering
`mermaid_version`	`"11"`	Major version of Mermaid.js to load from CDN

When enabled, pages containing `::: mermaid` blocks will load Mermaid.js and render diagrams client-side. When disabled, `::: mermaid` content renders as preformatted text.

`[locale]` Section

KEY	DEFAULT	DESCRIPTION
`default`	`None`	Default locale code (e.g. `"en"`). Required to enable i18n.
`enabled`	`[]`	List of enabled locale codes (e.g. `["en", "fr", "de"]`)
`auto_detect`	`true`	Auto-detect the user's browser language and redirect on first visit
`display_-names`	`{}`	Human-readable names for locales shown in the language switcher
`flags`	`{}`	Flag emoji overrides for locales (e.g. `{"en": "🇺🇸"}` to use US flag instead of default 🇬🇧)

When both `default` and `enabled` are set, DocAnvil switches to multi-language mode: each locale gets its own URL prefix (`/en/`, `/fr/`), its own navigation and search index, and a language switcher appears in the header.

```
[locale]
default = "en"
enabled = ["en", "fr", "de"]
auto_detect = true

[locale.display_names]
en = "English"
fr = "Français"
de = "Deutsch"

[locale.flags]
en = "🇺🇸"    # Use US flag instead of default GB
```

> Need details?
>
> See Localisation for a complete walkthrough of setting up multi-language docs, including file naming, per-locale navigation, and translation coverage.

`[pdf]` Section

KEY	DEFAULT	DESCRIPTION
`author`	`None`	Author name shown on the cover page and in the running page header
`cover_page`	`false`	Prepend a cover page with the project title and author before the table of contents
`paper_size`	`"A4"`	Paper size: `"A3"`, `"A4"`, `"A5"`, `"Letter"`, `"Legal"`, `"Tabloid"` (case-insensitive)
`custom_css`	`None`	Path (relative to project root) to a CSS file injected into the PDF output

> Need details?
>
> See PDF Export for the full guide: cover pages, paper sizes, RTL support, per-locale export, and custom CSS.

nav.toml

The navigation file controls the sidebar structure. It uses TOML's array-of-tables syntax and supports pages, separators, and groups.

Page Entries

The simplest entry links to a page by its slug (the file path relative to `content_dir`, without the `.md` extension):

```
[[nav]]
page = "index"

[[nav]]
page = "guides/getting-started"
```

Label Overrides

By default, the sidebar label is derived from the slug (`getting-started` becomes "Getting Started"). Override it with `label`:

```
[[nav]]
page = "guides/getting-started"
label = "Installation"
```

Separators

Add visual dividers between sections. A labeled separator shows text:

```
[[nav]]
separator = "Guides"
```

An unlabeled separator draws a horizontal line:

```
[[nav]]
separator = true
```

Groups

Groups create collapsible sections in the sidebar. Each group has a `label` and an array of children in `group`:

```
[[nav]]
label = "Reference"
group = [
  { page = "reference/cli", label = "CLI Commands" },
  { page = "reference/project-structure" },
  { page = "reference/css-variables", label = "CSS Variables" },
]
```

Linked Group Headers

Add a `page` field to make the group header itself a clickable link:

```
[[nav]]
label = "Writing Content"
page = "writing/markdown"
group = [
  { page = "writing/wiki-links", label = "Links & Popovers" },
  { page = "writing/components" },
]
```

Clicking "Writing Content" navigates to the Markdown page, while the arrow expands the group.

Child Separators

You can add separators inside groups to organize children:

```
[[nav]]
label = "Reference"
group = [
  { page = "reference/cli", label = "CLI Commands" },
  { separator = "Project" },
  { page = "reference/project-structure" },
  { page = "reference/css-variables", label = "CSS Variables" },
]
```

Autodiscover

You can use the autodiscover option to selectively autodiscover a folder and add it to the navigation:

```
[[nav]]
autodiscover = "api"
```

You can also use autodiscover with a collapsible group:

```
[[nav]]
label = "Reference"
autodiscover = "reference"
```

Auto-Discovery Fallback

If `nav.toml` is absent, DocAnvil auto-discovers all `.md` files in the content directory and builds the navigation from the directory structure. Files are sorted alphabetically, and directory names become group labels.

Related Pages

- Theming — CSS variables, custom CSS, and template overrides
- PDF Export — full PDF export guide
- Project Structure — how files map to pages and slugs

> Note
>
> The header includes a filter input that searches page labels in real time. This works with any navigation structure.

Installation

Get DocAnvil running and create your first documentation site.

Install DocAnvil

From crates.io | From GitHub

```
# Install from crates.io (when published)
cargo install docanvil
```

```
# Build from source
git clone https://github.com/docanvil/docanvil.git
cd docanvil
cargo install --path .
```

Verify the installation:

```
docanvil --help
```

Create a Project

Scaffold a new documentation project with `docanvil new`:

```
docanvil new my-docs
```

This creates the following structure:

```
my-docs/
  docanvil.toml        # Project configuration
  nav.toml             # Navigation structure
  docs/                # Your Markdown content
    index.md           # Home page
    guides/
      getting-started.md
      configuration.md
  theme/
    custom.css         # Your CSS overrides
```

Start the Dev Server

```
cd my-docs
docanvil serve
```

The dev server starts at http://localhost:3000 by default. You can change the host and port:

```
docanvil serve --host 0.0.0.0 --port 8080
```

Write Your First Page

Create a new Markdown file anywhere in the `docs/` directory:

```
# My New Page

Welcome to my documentation!

- Supports **bold**, *italic*, and ~~strikethrough~~
- Add links to other pages with wiki-link syntax
```

Save the file and your browser will reload automatically. The page is discovered and added to the navigation.

Build for Production

When you're ready to deploy, generate the static site:

```
docanvil build
```

The output goes to the `dist/` directory by default. Upload it to any static host — GitHub Pages, Netlify, Vercel, S3, or just a plain web server.

Use `--clean` to remove the output directory before building:

```
docanvil build --clean
```

For CI/CD pipelines, use `--strict` to fail the build when there are any warnings:

```
docanvil build --strict
```

Checklist

- [x] Install DocAnvil
- [x] Run `docanvil new` to scaffold a project
- [x] Start the dev server with `docanvil serve`
- [ ] Write your pages in Markdown
- [ ] Customize the theme
- [ ] Build and deploy with `docanvil build`

Next Steps

- Configure your project and navigation
- Learn about Markdown features and components
- Customize the theme to match your brand

> Note
>
> DocAnvil watches all files in your project directory. Changes to Markdown, config files, CSS, and templates all trigger a live reload.

Localisation

DocAnvil supports multi-language documentation sites out of the box. Each locale gets its own URL prefix, navigation, search index, and a language switcher in the header — all from a single content directory.

Enabling i18n

Add a `[locale]` section to your `docanvil.toml`:

```
[locale]
default = "en"
enabled = ["en", "fr"]
auto_detect = true

[locale.display_names]
en = "English"
fr = "Français"
```

That's it. DocAnvil will now look for locale suffixes in your filenames and build a separate site tree for each language.

File Naming Convention

Add the locale code as a suffix before the `.md` extension:

```
docs/
  index.en.md          # English home page
  index.fr.md          # French home page
  guides/
    getting-started.en.md
    getting-started.fr.md
    advanced.en.md      # No French translation (yet)
```

Files **without** a locale suffix are treated as the default locale. So if your default is `"en"`, then `index.md` and `index.en.md` are equivalent — both become the English version.

> No false positives
>
> Only suffixes that match your `enabled` list are recognized as locale codes. A file like `api.v2.md` won't be mistakenly treated as a locale — `v2` isn't in your enabled list.

Output Structure

Each locale gets its own directory in the build output:

```
dist/
  js/docanvil.js        # Shared assets (one copy)
  robots.txt
  sitemap.xml           # Contains all locales
  404.html              # Links to each locale's home page
  en/
    index.html
    guides/
      getting-started.html
    search-index.json   # English search index
  fr/
    index.html
    guides/
      getting-started.html
    search-index.json   # French search index
```

All locales get URL prefixes — even the default. This keeps URLs consistent and predictable.

Navigation

Auto-Discovered Nav

When you don't have a `nav.toml`, DocAnvil auto-discovers pages for each locale separately. The English nav only shows English pages, the French nav only shows French pages.

Per-Locale nav.toml

You can create locale-specific navigation files:

- `nav.fr.toml` — used for the French build
- `nav.en.toml` — used for the English build

- `nav.toml` — fallback for any locale without its own file

Slugs in nav files reference **base slugs** without the locale suffix. A page at `docs/guides/setup.en.md` has the base slug `guides/setup`, so your nav file uses:

```
[[nav]]
page = "guides/setup"
```

The same slug works in both the English and French nav files — DocAnvil resolves it to the correct locale's page.

Wiki-Links

Wiki-links resolve **within the same locale**. If you write `getting-started` in a French page, it links to the French version of that page. You don't need to add locale suffixes to your links.

This means your content files can be translated independently without updating any internal links.

Search

Each locale gets its own search index (`en/search-index.json`, `fr/search-index.json`). The search UI automatically loads the right index for the current locale, so users only see results in their language.

Language Switcher

When i18n is enabled, a language switcher appears in the header bar. It shows:

- A flag emoji with the current locale code (e.g. 🇬🇧 EN)
- A dropdown listing all enabled locales with flag emoji and display names
- The current locale highlighted
- Unavailable translations greyed out (when a page doesn't exist in that locale)

Clicking a locale navigates to the same page in the selected language. If the page doesn't exist in the target locale, the link goes to that locale's home page instead.

Flag Emoji

DocAnvil auto-assigns flag emoji based on locale codes — `en` gets 🇬🇧, `fr` gets 🇫🇷, `de` gets 🇩🇪, and so on. Unknown locale codes get a 🌐 globe.

To override the default flag for a locale (e.g. using the US flag for English), add a `[locale.flags]` table:

```
[locale.flags]
en = "🇺🇸"
```

This is useful when a language maps to multiple countries and you want to match your audience.

Browser Auto-Detection

When `auto_detect` is `true` (the default), DocAnvil checks the visitor's browser language on their first visit:

1. If the user has previously chosen a language (stored in `localStorage`), that choice is respected
2. If not, the browser's `navigator.language` is checked against the enabled locales
3. If it matches a different locale than the current page, the user is redirected
4. The detected language is saved to `localStorage` to prevent repeated redirects

Set `auto_detect = false` to disable this behavior entirely.

Missing Translations

When a page exists in some locales but not all, DocAnvil emits a build warning:

```
warning: page 'guides/advanced' has no translation for locale 'fr'
  hint: Create a file with the '.fr.md' suffix to add a translation.
```

In `--strict` mode, these warnings become errors and the build fails. This is useful in CI to enforce complete translations before deploying.

The `docanvil doctor` command also checks translation coverage when i18n is enabled, reporting:

- **missing-translation** (Warning) — pages missing in some locales

- **orphaned-locale** (Warning) — files with locale suffixes not in the enabled list
- **missing-default-locale** (Error) — default locale has no pages at all

SEO

DocAnvil generates comprehensive multilingual SEO signals when i18n is enabled:

hreflang Tags

Each page includes `<link rel="alternate" hreflang="...">` tags pointing to every available translation. These tell search engines which pages are translations of each other, preventing duplicate content issues and ensuring users see results in their language.

An `hreflang="x-default"` tag is also emitted, pointing to the default locale's version of the page. This acts as a fallback for users whose language isn't in your enabled list.

Canonical URLs and Open Graph

When `site_url` is configured, each page gets:

- `<link rel="canonical">` — the definitive URL for the page
- `<meta property="og:url">` — the URL used when the page is shared on social media
- `<meta property="og:locale">` — the current page's locale
- `<meta property="og:locale:alternate">` — tags for each translation

These tags use absolute URLs derived from `site_url`. Without `site_url`, hreflang tags still work with relative URLs, but canonical and og:url are omitted.

Sitemap

The sitemap includes all pages across all locales with `xhtml:link` hreflang annotations:

```
<url>
  <loc>https://example.com/en/guides/setup.html</loc>
  <xhtml:link rel="alternate" hreflang="en" href="https://example.com/en/guides/s
  <xhtml:link rel="alternate" hreflang="fr" href="https://example.com/fr/guides/s
  <xhtml:link rel="alternate" hreflang="x-default" href="https://example.com/en/g
</url>
```

Google recommends both in-page hreflang tags and sitemap hreflang annotations — DocAnvil does both automatically.

HTML lang Attribute

The `<html>` tag includes a `lang` attribute matching the current locale, which helps search engines and screen readers.

> Tip
>
> Set `site_url` in your `docanvil.toml` to get the most out of multilingual SEO. Without it, canonical URLs and absolute hreflang links can't be generated.

Backward Compatibility

When no `[locale]` section exists in `docanvil.toml`:

- Pages build to the root output directory (no locale prefixes)
- No language switcher appears
- Wiki-links resolve globally as before
- Search index lives at `/search-index.json`
- Doctor skips translation checks

Existing single-language projects work without any changes.

Related Pages

- Configuration — `[locale]` config options
- Links & Popovers — how wiki-links resolve within locales
- Project Structure — i18n directory layout
- CLI Commands — `--strict` mode and `docanvil doctor`

Migration Guide

DocAnvil is pre-1.0, so breaking changes can happen between minor versions. This page will document migration steps as they come up.

Migrating Between Versions

Nothing to migrate yet! DocAnvil hasn't introduced any breaking changes that require manual intervention.

When breaking changes do land, you'll find step-by-step migration instructions here — covering config changes, renamed options, and anything else that might affect your project.

Coming From Another Tool

If you're moving an existing documentation site to DocAnvil, the process is straightforward:

1. Run `docanvil new my-docs` to scaffold a fresh project
2. Copy your Markdown files into the `docs/` directory
3. Set up your navigation in `nav.toml` (or let auto-discovery handle it)
4. Adjust any tool-specific syntax (front matter format, component directives, etc.)

DocAnvil uses JSON front matter (not YAML), so you may need to convert front matter blocks. See Front Matter for the format.

> Need help migrating?
>
> If you run into issues moving from another documentation tool, open a discussion on GitHub — we'd love to hear what tripped you up so we can make the process smoother.

PDF Export

DocAnvil can export your entire documentation site as a single, print-ready PDF using Chrome or Chromium — no external tooling, no Python, no Pandoc.

The exported PDF includes a table of contents, all your pages in navigation order, syntax-highlighted code blocks, Mermaid diagrams, and an optional cover page. Running headers and page numbers are added automatically.

Prerequisites

PDF export requires **Google Chrome** or **Chromium** to be installed. DocAnvil launches the browser in headless mode and uses the Chrome DevTools Protocol (CDP) to render the page and print it to PDF.

DocAnvil searches for Chrome in these locations (in order):

macOS Windows Linux

- `/Applications/Google Chrome.app/Contents/MacOS/Google Chrome`
- `chromium` on PATH

- `%ProgramFiles%\Google\Chrome\Application\chrome.exe`
- `%ProgramFiles(x86)%\Google\Chrome\Application\chrome.exe`

- `google-chrome`, `google-chrome-stable`
- `chromium-browser`, `chromium`, `chrome`
- Any of the above on PATH

If Chrome isn't found, the command exits with a clear error message.

Basic Usage

```
docanvil export pdf --out guide.pdf
```

OPTION	REQUIRED	DEFAULT	DESCRIPTION
`--out`	Yes	—	Output path for the PDF file
`--path`	No	`.`	Path to the project root
`--locale`	No	project default	Locale to export. Pass `all` to generate one PDF per enabled locale.

Parent directories for the output path are created automatically.

bash bash bash

```bash
# Export for the current project
docanvil export pdf --out guide.pdf
```

```bash
# Export a project in another directory
docanvil export pdf --out guide.pdf --path ../my-docs
```

```bash
# Suppress progress output (useful in scripts)
docanvil export pdf --out guide.pdf --quiet
```

Configuration

PDF export is configured in the `[pdf]` section of `docanvil.toml`:

```toml
[pdf]
author = "Your Name"
cover_page = true
paper_size = "A4"
custom_css = "theme/pdf.css"
```

KEY	DEFAULT	DESCRIPTION
`author`	`None`	Author name shown on the cover page and in the running page header
`cover_page`	`false`	Prepend a cover page with the project title and author name
`paper_size`	`"A4"`	Paper size — see Custom Paper Size for supported values
`custom_css`	`None`	Path (relative to the project root) to a CSS file injected into the PDF

Cover Page

When `cover_page = true`, a title page is added before the table of contents. It shows the project name as a large centered title with the author below it.

```
[pdf]
cover_page = true
author = "The DocAnvil Team"
```

Custom Paper Size

```
[pdf]
paper_size = "Letter"
```

Supported sizes (case-insensitive):

SIZE	DIMENSIONS
`A3`	297 × 420 mm (11.69 × 16.54 in)
`A4`	210 × 297 mm (8.27 × 11.69 in) — **default**
`A5`	148 × 210 mm (5.83 × 8.27 in)
`Letter`	8.5 × 11 in
`Legal`	8.5 × 14 in
`Tabloid`	11 × 17 in

Unrecognised values fall back to A4 silently.

Per-Locale Export

If your project uses localisation, you can generate a separate PDF for each enabled locale in a single command:

```
# One PDF per locale — locale code inserted before the extension
docanvil export pdf --out guide.pdf --locale all
# → guide.en.pdf, guide.fr.pdf, guide.de.pdf …
```

You can also export a single specific locale:

```
docanvil export pdf --out guide-fr.pdf --locale fr
```

Each PDF uses the locale's own navigation order and page content. Pages without a translation for that locale are silently skipped.

> Note
>
> `--locale all` requires i18n to be configured (`[locale]` with both `default` and `enabled` set in `docanvil.toml`). Running it on a non-i18n project returns a clear error.

RTL Language Support

Right-to-left locales are detected automatically. When you export in an RTL locale, Chrome lays out the entire PDF right-to-left — no configuration required.

```
docanvil export pdf --out guide-ar.pdf --locale ar
```

Supported RTL locale codes: `ar` (Arabic), `he` (Hebrew), `ur` (Urdu), `fa` (Persian/Farsi), `ug` (Uyghur).

Custom PDF CSS

For fine-grained control over the PDF's appearance, provide a CSS file:

```
[pdf]
custom_css = "theme/pdf.css"
```

The file is injected after the default PDF styles, so any rule you write overrides the defaults. Some useful targets:

```
/* Swap the typeface */
body {
  font-family: "Source Serif 4", serif;
  font-size: 10.5pt;
}

/* Tighter code blocks */
pre {
  font-size: 8.5pt;
}

/* Remove link colouring in print */
a {
  color: inherit;
  text-decoration: none;
}

/* Wider margins */
@page {
  margin: 3cm;
}
```

Mermaid Diagrams

`::: mermaid` blocks are rendered to SVG before the PDF is captured. DocAnvil waits up to 15 seconds for all diagrams to finish. If any are still pending after the timeout, the PDF is generated with whatever has rendered.

Mermaid.js is loaded from CDN, so Mermaid rendering requires an internet connection. In offline CI environments, disable charts:

```
[charts]
enabled = false
```

Running Headers and Footers

Every page gets:

- **Header left** — project name
- **Header right** — author name (if `author` is configured)
- **Footer right** — page number

These are injected by Chrome's print engine and are not affected by custom CSS.

Tips

- Run `docanvil build` first to confirm your content is error-free. Broken wiki-links and rendering issues will appear in the PDF just as they do in the HTML site.
- The PDF follows your `nav.toml` order exactly — the table of contents and chapter sequence match what readers see online.
- Use `--quiet` to suppress progress output in automated scripts.

Related Pages

- Localisation — setting up multi-language docs
- Configuration — full `docanvil.toml` reference including `[pdf]`
- CLI Commands — all subcommands and flags

Theming

DocAnvil's appearance is customizable through three layers, each building on the last:

1. **CSS variable overrides** in `docanvil.toml` — quick color and font changes
2. **Custom CSS file** — full control over any element
3. **Template overrides** — replace the entire HTML layout with Tera templates

Quick Start: Theme Generator

The fastest way to customize your site's colors is the interactive theme generator:

```
docanvil theme
```

This prompts for a primary and secondary color, then generates a complete `theme/custom.css` with all derived color variables and updates your `docanvil.toml` automatically. Run `docanvil serve` afterward to preview the result.

See CLI Commands for the full list of options (`--overwrite`, `--path`).

Dark Mode

DocAnvil supports light mode, dark mode, or both with an automatic toggle. Set the `color_mode` in your `docanvil.toml`:

```
[theme]
color_mode = "both"  # "light" (default) | "dark" | "both"
```

MODE	BEHAVIOR
`light`	Light palette only (default, current behavior)
`dark`	Dark palette only — dark backgrounds, light text
`both`	Light as default, with a sun/moon toggle in the header and OS `prefers-color-scheme` auto-detection

Using the Theme Generator

The easiest way to set up dark mode is through the theme generator:

```
docanvil theme
```

Select "Both (light + dark with toggle)" when prompted for color mode. You'll be asked for separate primary and secondary colors for each mode, and the generator will produce a single `theme/custom.css` with light variables in `:root`, dark variables in `[data-theme="dark"]`, and an `@media (prefers-color-scheme: dark)` block for OS auto-detection.

How the Toggle Works

When `color_mode = "both"`:

- A sun/moon icon button appears in the header
- On first visit, the OS preference is respected via `prefers-color-scheme`
- Clicking the toggle switches between light and dark and saves the choice to `localStorage`
- The choice persists across page navigations and browser sessions
- A flash-prevention script in `<head>` ensures the page renders in the correct mode immediately

Manual Dark Mode CSS

If you prefer to write your own dark mode CSS instead of using the generator, structure your `theme/custom.css` like this:

```
/* Light mode */
:root {
  --color-primary: #6366f1;
  /* ... other light variables ... */
}

/* Dark mode — explicit toggle */
[data-theme="dark"] {
  --color-bg: #0f172a;
  --color-text: #f1f5f9;
  /* ... other dark variables ... */
}

/* Dark mode — OS preference */
@media (prefers-color-scheme: dark) {
  :root:not([data-theme="light"]) {
    --color-bg: #0f172a;
    --color-text: #f1f5f9;
    /* ... same dark variables ... */
  }
}
```

The `[data-theme="dark"]` selector handles explicit user choice via the toggle, while the `@media` block handles OS-level preference when no explicit choice has been made.

CSS Variables in Config

The simplest way to customize the theme. Add variables under `[theme.variables]` in `docanvil.toml`:

```
[theme.variables]
color-primary = "#059669"
color-primary-light = "#34d399"
color-link = "#059669"
color-link-hover = "#047857"
```

These are injected as a `:root` style block after the default theme, overriding the built-in values. Variable names omit the `--` prefix — DocAnvil adds it automatically.

Custom CSS File

For more control, point `custom_css` to a stylesheet:

```
[theme]
custom_css = "theme/custom.css"
```

This file loads after both the default theme and config variable overrides, so it has the highest CSS specificity. Use it for:

- Additional variable overrides in a `:root` block
- Custom selectors targeting specific elements
- New styles for your own classes (via inline attributes)

Common Customizations

toml | css

```toml
# docanvil.toml — change accent color and font
[theme.variables]
color-primary = "#059669"
color-primary-light = "#34d399"
color-link = "#059669"
font-body = "Georgia, serif"
```

```
/* theme/custom.css — wider content area with dark code blocks */
.content {
  max-width: 960px;
}
<p>.content pre {
background: #1e293b;
color: #e2e8f0;
border-color: #334155;
}</p>
<p>.content pre code {
color: inherit;
}
```

Load Order

Styles are applied in this order (last wins):

1. Default theme (`style.css` embedded in the binary)
2. Config variables (`[theme.variables]` → `:root { ... }`)
3. Custom CSS file (`custom_css` path)

> Specificity
>
> If a custom CSS rule doesn't seem to take effect, check that your selector is specific enough to override the default theme. Using `.content pre` is more specific than just `pre`.

Template Overrides

For complete control over the HTML structure, override the default Tera template. Create a file at `theme/templates/layout.html` in your project:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>{{ page_title }} — {{ project_name }}</title>
  <style>{{ default_css | safe }}</style>
  {% if css_overrides %}
  <style>:root { {{ css_overrides }} }</style>
  {% endif %}
  {% if custom_css_path %}
  <link rel="stylesheet" href="/{{ custom_css_path }}">
  {% endif %}
  {% block head %}{% endblock %}
</head>
<body>
  {% block sidebar %}
  <nav class="sidebar">
    <div class="project-name">{{ project_name }}</div>
    {{ nav_html | safe }}
  </nav>
  {% endblock %}

  <main class="content">
    {% block content %}
    {{ content | safe }}
    {% endblock %}

    {% block footer %}
    <div class="footer">
      Built with <a href="https://github.com/docanvil/docanvil">DocAnvil</a>
    </div>
    {% endblock %}
  </main>

  {% block scripts %}{% endblock %}
</body>
</html>
```

Template Blocks

BLOCK	PURPOSE
`head`	Extra `<head>` content (fonts, meta tags, analytics)
`header`	Top header bar with project name and search
`sidebar`	The navigation sidebar
`content`	Main page content area
`footer`	Footer below content
`scripts`	JavaScript at end of body

Template Variables

VARIABLE	TYPE	DESCRIPTION
`page_title`	String	Title of the current page
`project_name`	String	Project name from `docanvil.toml`
`default_css`	String	The full default stylesheet (use with `safe` filter)
`css_overrides`	String	CSS variable overrides from config
`custom_css_path`	String	Path to custom CSS file, if configured
`nav_html`	String	Rendered navigation HTML (use with `safe` filter)
`content`	String	Rendered page HTML (use with `safe` filter)
`live_reload`	Boolean	Whether the dev server is running
`search_enabled`	Boolean	Whether full-text search is enabled
`mermaid_enabled`	Boolean	Whether Mermaid diagram rendering is enabled
`mermaid_version`	String	Mermaid.js major version to load from CDN
`color_mode`	String	Color mode: `"light"`, `"dark"`, or `"both"`

> Note
>
> The default template includes JavaScript for tab switching, sidebar collapse/expand, navigation filtering, popover positioning, search, and Mermaid diagram rendering. If you override the `scripts` block, you'll need to re-implement any of these features you want to keep.

Related Pages

- CSS Variables — complete list of every variable and its default value
- Configuration — `docanvil.toml` and `nav.toml` reference

Components

DocAnvil provides built-in components rendered via fenced directives. These are processed before Markdown rendering, so you can use Markdown inside them.

Directive Syntax

Components use a fenced block syntax with triple colons:

```
<div class="name">
<p>Content goes here. <strong>Markdown</strong> is supported.</p>
</div>
```

The opening fence is `:::` followed by the component name and optional attributes in curly braces. The closing fence is `:::` on its own line.

Attributes

Attributes are specified inside `{...}` after the component name:

SYNTAX	RESULT
`key="value"`	Named attribute
`.classname`	Adds a CSS class
`#idname`	Sets the element ID

Multiple attributes can be combined: `::: note{title="Important" .custom-class #my-note}`

Note

Display informational callouts with a blue/indigo theme.

> Note
>
> This is a note with the default title.

> Custom Title
>
> Notes accept a `title` attribute. The default title is "Note".

Raw syntax:

```
<div class="admonition note">
<p class="admonition-title">Custom Title</p>
<p>Your content here. Supports <strong>Markdown</strong>.</p>
</div>
```

Warning

Display cautionary messages with an orange theme.

> Warning
>
> This is a warning with the default title.

> Danger Zone
>
> Warnings accept a `title` attribute. The default title is "Warning".

Raw syntax:

```
<div class="admonition warning">
<p class="admonition-title">Danger Zone</p>
<p>Your warning content here.</p>
</div>
```

Lozenge

Display a quick visual status with a lozenge.

Syntax is as follow: `::: lozenge{type="default",text="Default"}`

SYNTAX	RESULT
`DEFAULT`	Default
`WARNING`	Warning
`IN PROGRESS`	In Progress
`ERROR`	Error
`SUCCESS`	Success

Tabs

Group content into switchable tabs. Each tab is defined with a nested `::: tab` directive. The outer `::::tabs` uses four colons so the inner `::: tab` closings (three colons) don't end the container prematurely:

JavaScript | Python | Rust

```
console.log("Hello!");
```

```
print("Hello!")
```

```
fn main() {
    println!("Hello!");
}
```

Raw syntax:

```
<div class="tabs">
<div class="tab-headers">
  <button class="tab-header active" data-tab="0">JavaScript</button>
  <button class="tab-header" data-tab="1">Python</button>
</div>
<div class="tab-content active" data-tab="0">
<p>Content for the JavaScript tab.</p>
</div>
<div class="tab-content" data-tab="1">
<p>Content for the Python tab.</p>
</div>
</div>
```

If no `title` is provided, tabs are labeled "Tab 1", "Tab 2", etc.

Code Group

A specialized tab component for comparing code blocks across languages. Each fenced code block becomes a tab, with the language name as the tab label:

rust python javascript

```rust
fn greet(name: &str) {
    println!("Hello, {name}!");
}
```

```python
def greet(name):
    print(f"Hello, {name}!")
```

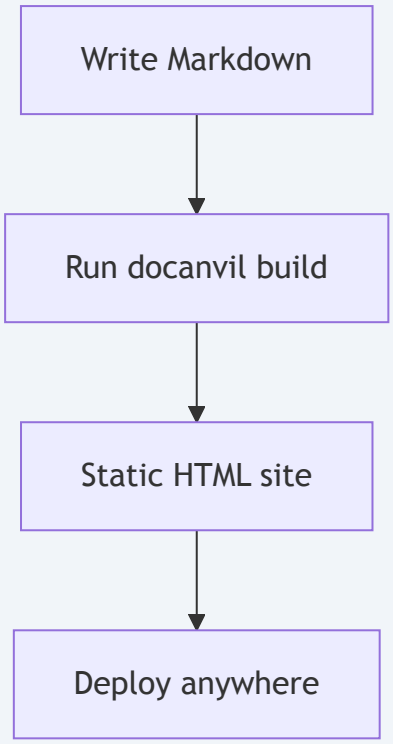
```javascript
function greet(name) {
  console.log(`Hello, ${name}!`);
}
```

Raw syntax:

```
<div class="code-group">
<div class="tab-headers">
  <button class="tab-header active" data-tab="0">rust</button>
  <button class="tab-header" data-tab="1">python</button>
</div>
<div class="tab-content active" data-tab="0"><pre><code class="language-rust">fn
    println!("Hello, {name}!");
}</code></pre></div>
<div class="tab-content" data-tab="1"><pre><code class="language-python">def gree
    print(f"Hello, {name}!")</code></pre></div>
</div>
```

Mermaid Diagrams

Render diagrams and charts using Mermaid.js. The content inside a `::: mermaid` block is passed directly to Mermaid — it is not processed as Markdown.



Raw syntax:

```
<pre class="mermaid">graph TD
    A[Write Markdown] --> B[Run docanvil build]
    B --> C[Static HTML site]
    C --> D[Deploy anywhere]</pre>
```

Mermaid supports many diagram types including flowcharts, sequence diagrams, class diagrams, state diagrams, Gantt charts, and more. See the Mermaid documentation for the full syntax reference.

> Configuration
>
> Mermaid is enabled by default. Disable it by setting `enabled = false` under `[charts]` in `docanvil.toml`. When disabled, `::: mermaid` blocks render as preformatted text. See Configuration for details.

Nesting Directives

When nesting directives, use more colons on the outer fence to distinguish it from inner closings. The `::::tabs` (four colons) and `::: tab` (three colons) pattern is the primary example of this:

```
<div class="tabs">
<div class="tab-headers">
  <button class="tab-header active" data-tab="0">First</button>
  <button class="tab-header" data-tab="1">Second</button>
</div>
<div class="tab-content active" data-tab="0">
<p>Content here.</p>
</div>
<div class="tab-content" data-tab="1">
<p>Content here.</p>
</div>
</div>
```

The outer directive uses 4 colons (`::::`) while the inner ones use 3 (`:::`). The closing fence must match the exact number of colons used in the opening fence.

Unknown Directives

If you use a directive name that doesn't match a built-in component, the content is wrapped in a `<div>` with the directive name as the class:

```
<div class="custom-block">
<p>This becomes a <code><div class="custom-block extra"></code>.</p>
</div>
```

This lets you create custom styled blocks using your own CSS.

Summary

COMPONENT	DIRECTIVE	KEY ATTRIBUTE	DEFAULT
Note	`::: note`	`title`	`"Note"`
Warning	`::: warning`	`title`	`"Warning"`
Tabs	`::::tabs` + `::: tab`	`title` (on tab)	`"Tab 1"`, `"Tab 2"`, ...
Code Group	`::: code-group`	*(none)*	Language name from code fence
Mermaid	`::: mermaid`	*(none)*	Renders diagram via Mermaid.js

> Note
>
> Components are processed before Markdown rendering. This means you can use bold, italic, links, code, and other Markdown formatting inside any component.

Related Pages

- Markdown — text formatting, tables, and other Markdown features
- Wiki-links — double-bracket links and inline popovers

Front Matter

Front matter is a block of JSON metadata at the top of a Markdown file, wrapped in `---` delimiters. DocAnvil parses front matter and uses it to set page titles, generate SEO meta tags, and populate Open Graph metadata in the HTML output.

Basic Syntax

Place a JSON block at the very start of your Markdown file:

```
---
{
  "title": "Getting Started",
  "description": "Learn how to install and configure DocAnvil",
  "author": "Jane Doe",
  "date": "2024-01-15"
}
---

Your page content starts here.
```

The front matter block is stripped from the rendered output — it only affects metadata.

Supported Fields

All fields are optional. You can include any combination of them or omit front matter entirely.

FIELD	TYPE	EFFECT
`title`	String	Overrides the page title used in the browser tab, navigation sidebar, search index, breadcrumbs, and URL slug
`slug`	String	Overrides the URL slug directly — takes priority over the title-derived slug
`description`	String	Renders as `<meta name="description">` and `<meta property="og:description">` for search engines and link previews
`author`	String	Renders as `<meta name="author">`
`date`	String	Renders as `<meta property="article:published_time">` for search engines and social sharing

Unknown fields are silently ignored, so you can add your own custom metadata without causing errors.

Title Override

By default, DocAnvil derives page titles from filenames — `getting-started.md` becomes "Getting Started". Front matter `title` overrides this everywhere:

- The `<title>` tag in the HTML head
- The navigation sidebar label
- The search index
- Breadcrumb trails
- The URL slug and output filename

```
---
{
  "title": "Quick Start Guide"
}
---

# Getting Started with DocAnvil

Content here...
```

In this example, the sidebar and browser tab show "Quick Start Guide" while the page content displays its own `# Getting Started with DocAnvil` heading.

Clean URLs from Titles

When a `title` is set, the page's URL slug is derived from the title instead of the filename. This is especially useful for files with organizational prefixes:

FILENAME	TITLE	OUTPUT URL
`01-introduction.md`	`"Introduction"`	`/introduction.html`
`03-setup-guide.md`	`"Setup Guide"`	`/setup-guide.html`
`guides/01-basics.md`	`"The Basics"`	`/guides/the-basics.html`

The directory prefix is always preserved — only the filename portion changes.

> Index pages are exempt
>
> Pages named `index.md` keep their slug regardless of the `title` field. The `index` URL is a well-known convention and is never overridden by title. Use the explicit `slug` field if you need to change it.

Slug Override

For full control over the output URL, use the `slug` field. It takes priority over both the filename and the title-derived slug.

```
---
{
  "title": "Getting Started with DocAnvil",
  "slug": "quickstart"
}
---
```

This page will be written to `/quickstart.html` while still displaying "Getting Started with DocAnvil" as the page title.

The `slug` value is normalized to a URL-safe format automatically — spaces become hyphens and special characters are removed.

Backward-Compatible Links

When a slug changes (via `title` or `slug`), wiki-links using the old filename-based slug still resolve correctly. For example, if `01-setup.md` gets the title "Setup Guide", both `01-setup` and `setup-guide` will link to the same page.

SEO Meta Tags

When front matter fields are present, DocAnvil generates the corresponding HTML meta tags in the page `<head>`:

```
<meta name="description" content="Learn how to install and configure DocAnvil">
<meta property="og:description" content="Learn how to install and configure DocAn
<meta name="author" content="Jane Doe">
<meta property="article:published_time" content="2024-01-15">
```

Every page also gets these Open Graph tags automatically, regardless of front matter:

```
<meta property="og:title" content="Getting Started">
<meta property="og:type" content="article">
```

Examples

Minimal — title only

```
---
{
  "title": "API Reference"
}
---
```

Full metadata

```
---
{
  "title": "Deployment Guide",
  "description": "Deploy your DocAnvil site to Netlify, Vercel, or GitHub Pages",
  "author": "Documentation Team",
  "date": "2024-06-01"
}
---
```

Custom slug

```
---
{
  "title": "Frequently Asked Questions",
  "slug": "faq"
}
---
```

This outputs to `/faq.html` instead of `/frequently-asked-questions.html`.

No front matter

Pages without front matter work exactly as before — the title is derived from the filename and no extra meta tags are added.

Date Format

The `date` field is passed through as-is to the `article:published_time` meta tag. ISO 8601 format (`YYYY-MM-DD`) is recommended for best compatibility with search engines and social platforms.

Markdown

DocAnvil renders Markdown using comrak with GitHub Flavored Markdown (GFM) extensions enabled. Everything you'd expect from standard Markdown works, plus tables, task lists, strikethrough, footnotes, and front matter.

Text Formatting

Bold text is wrapped in double asterisks: `**bold**`

Italic text uses single asterisks: `*italic*`

~~Strikethrough~~ uses double tildes: `~~strikethrough~~`

You can combine them: ***bold and italic***, ~~**bold strikethrough**~~

Highlighted text uses double equals: `==highlighted==`

Superscript uses carets: X^2 is written as `X^2^`

Subscript uses single tildes: H_2O is written as `H~2~O`

Emoji shortcodes like `:smile:` are converted to their Unicode equivalents: 😄

Headings

```
# Heading 1
## Heading 2
### Heading 3
#### Heading 4
```

Heading 1 gets a colored bottom border. Heading 2 gets a subtle separator line. Headings 3 and 4 are unstyled dividers.

Custom Heading IDs

By default, headings get auto-generated IDs based on their text. You can override this with a custom ID using `{#id}` at the end of the heading line:

```
### API Reference

{#api-ref}
```

This renders as `<h3 id="api-ref">API Reference</h3>` instead of the auto-generated `api-reference`.

Links and Images

Standard Markdown links: `[text](url)`

Images: `![alt text](image-url)`

For linking between documentation pages, use wiki-links instead — they resolve automatically and warn on broken references.

Lists

Unordered

- First item
- Second item
  - Nested item
  - Another nested item
- Third item

Ordered

1. Step one
2. Step two
3. Step three
   1. Sub-step
   2. Another sub-step

Task Lists

- [x] Write the documentation
- [x] Add code examples
- [ ] Review and publish

- [ ] Celebrate

Task lists render as checkboxes. Use `- [x]` for checked and `- [ ]` for unchecked items.

Tables

FEATURE	SYNTAX	RENDERED
Bold	`**text**`	**text**
Italic	`*text*`	*text*
Code	`` `code` ``	`code`
Strikethrough	`~~text~~`	~~text~~

Tables support column alignment with colons in the separator row:

```
| Left | Center | Right |
|:-----|:------:|------:|
| a    |   b    |     c |
```

Code Blocks

Inline code uses single backticks: `let x = 42;`

Fenced code blocks use triple backticks with an optional language identifier:

```
fn main() {
    println!("Hello from DocAnvil!");
}
```

```
const greet = (name) => {
  console.log(`Hello, ${name}!`);
};
```

Blockquotes

> *Blockquotes are rendered with a colored left border and a subtle background.*
>
> *They can span multiple paragraphs.*

Use `>` at the start of each line.

Footnotes

DocAnvil supports footnotes[1] using the standard syntax. Reference them inline with `[^name]` and define them anywhere in the document.

Here's another example with a longer footnote[2].

Front Matter

Pages can include JSON front matter between `---` delimiters at the top of the file. Front matter lets you set custom page titles, descriptions, author info, and dates — which DocAnvil uses for navigation labels, search, and SEO meta tags.

```
---
{
  "title": "My Page Title",
  "description": "A brief summary for search engines",
  "author": "Jane Doe",
  "date": "2024-01-15"
}
---

# Page Content
```

See Front Matter for the full list of supported fields and examples.

Horizontal Rules

Three or more hyphens, asterisks, or underscores create a horizontal rule:

```
---
```

Inline Attributes

DocAnvil supports a post-processing pass for inline attributes. Place `{.class #id}` on the line immediately after an element to inject HTML attributes:

```
## My Section
{#custom-id .highlighted}
```

This renders as `<h2 id="custom-id" class="highlighted">My Section</h2>`.

Supported shorthand:

SYNTAX	RESULT
`.classname`	`class="classname"`
`#idname`	`id="idname"`
`key="value"`	`key="value"`

Multiple classes can be combined: `{.first .second #my-id}`

Related Pages

- Front Matter — page metadata, titles, and SEO meta tags
- Wiki-links — double-bracket links and inline popovers
- Components — notes, warnings, tabs, and code groups

> Powered by comrak
>
> DocAnvil uses comrak for Markdown rendering with `unsafe` mode enabled, so raw HTML in your Markdown is passed through to the output.

1. This is a footnote. It appears at the bottom of the page in a dedicated section. ↩

2. Footnotes can contain multiple sentences. They're collected and rendered at the bottom of the page with a horizontal separator and back-references. ↩

CSS Variables

DocAnvil's default theme is built entirely on CSS custom properties (variables). Override any of them to customize the look of your site without writing complex CSS.

How to Override

docanvil.toml custom.css

Set variables in the `[theme.variables]` section. Omit the `--` prefix:

```
[theme.variables]
color-primary = "#059669"
font-body = "Georgia, serif"
content-max-width = "960px"
```

Override variables in a `:root` block in your custom CSS file:

```
:root {
  --color-primary: #059669;
  --font-body: Georgia, serif;
  --content-max-width: 960px;
}
```

Colors

VARIABLE	DEFAULT	DESCRIPTION
`--color-primary`	`#6366f1`	Primary accent color (links, active states, borders)
`--color-primary-light`	`#818cf8`	Lighter primary variant (h1 border, blockquote border)
`--color-bg`	`#ffffff`	Page background
`--color-bg-secondary`	`#f8fafc`	Secondary background (sidebar, table headers, blockquotes)
`--color-text`	`#1e293b`	Main text color
`--color-text-muted`	`#64748b`	Muted text (headings h4, separators, footer)
`--color-border`	`#e2e8f0`	Border color used throughout
`--color-link`	`#6366f1`	Link text color
`--color-link-hover`	`#4f46e5`	Link hover color
`--color-code-bg`	`#f1f5f9`	Background for inline code and code blocks
`--color-note-bg`	`#eef2ff`	Note admonition background
`--color-note-border`	`#818cf8`	Note admonition left border
`--color-warning-bg`	`#fff7ed`	Warning admonition background
`--color-warning-border`	`#f97316`	Warning admonition left border

Typography

VARIABLE	DEFAULT	DESCRIPTION
`--font-body`	`system-ui, -apple-system, "Segoe UI", Roboto, sans-serif`	Body font stack
`--font-mono`	`"SF Mono", Consolas, "Liberation Mono", Menlo, monospace`	Monospace font for code
`--font-size-base`	`16px`	Base font size
`--font-size-sm`	`0.875rem`	Small font size (table headers, footnotes)
`--line-height-tight`	`1.3`	Tight line height for headings
`--heading-letter-spacing`	`-0.02em`	Letter spacing for headings

Layout

VARIABLE	DEFAULT	DESCRIPTION
`--sidebar-width`	`260px`	Width of the navigation sidebar
`--content-max-width`	`800px`	Maximum width of the content area

Sidebar

VARIABLE	DEFAULT	DESCRIPTION
`--color-sidebar-hover`	`#eef2ff`	Background on sidebar link hover
`--color-sidebar-active-bg`	`#eef2ff`	Background of the active sidebar link
`--color-sidebar-active-text`	`#4f46e5`	Text color of the active sidebar link

Navigation

VARIABLE	DEFAULT	DESCRIPTION
`--nav-filter-bg`	`#ffffff`	Background of the filter input
`--nav-filter-border`	`var(--color-border)`	Border color of the filter input
`--nav-group-toggle-hover`	`rgba(99, 102, 241, 0.06)`	Background on group toggle hover

Shadows

VARIABLE	DEFAULT	DESCRIPTION
`--shadow-sm`	`0 1px 2px rgba(0, 0, 0, 0.05)`	Small shadow (code blocks, admonitions)
`--shadow-md`	`0 4px 6px -1px rgba(0, 0, 0, 0.07), 0 2px 4px -2px rgba(0, 0, 0, 0.05)`	Medium shadow (popovers, images)

Border Radius

VARIABLE	DEFAULT	DESCRIPTION
`--radius-sm`	`4px`	Small radius (inline code, nav items, filter input)
`--radius-md`	`6px`	Medium radius (code blocks, tables, popovers)
`--radius-lg`	`8px`	Large radius (admonitions)

Transitions

VARIABLE	DEFAULT	DESCRIPTION
`--transition-fast`	`150ms ease`	Fast transitions (hover states)
`--transition-normal`	`200ms ease`	Normal transitions (chevron rotation)

Focus

VARIABLE	DEFAULT	DESCRIPTION
`--color-focus-ring`	`rgba(99, 102, 241, 0.4)`	Focus ring color for keyboard navigation

> Note
>
> All color values use hex or rgba notation. Font stacks use standard CSS syntax with fallbacks. Sizes accept any CSS length unit (px, rem, em).

Related Pages

- Theming — how to apply variable overrides and custom CSS
- Configuration — `docanvil.toml` reference

Project Structure

A DocAnvil project has a simple directory layout. Here's the complete structure:

```
my-project/
  docanvil.toml           # Project configuration
  nav.toml                # Navigation structure (optional)
  docs/                   # Content directory (configurable)
    index.md              # Home page
    guides/
      getting-started.md
      configuration.md
    reference/
      cli.md
  theme/                  # Theme customization
    custom.css            # Custom stylesheet
    templates/            # Template overrides (optional)
      layout.html
  dist/                   # Build output (generated)
    index.html
    search-index.json     # Full-text search index (when search enabled)
    guides/
      getting-started.html
    theme/
      custom.css
```

Files and Directories

`docanvil.toml`

The main configuration file. Contains project name, content directory path, build output path, and theme settings. See Configuration for the full reference.

`nav.toml`

Controls the sidebar navigation structure. Defines page order, separators, groups, and label overrides. If this file is absent, DocAnvil auto-discovers pages from the content directory and builds navigation alphabetically.

Content Directory

The `docs/` directory (configurable via `content_dir` in `docanvil.toml`) contains all your Markdown source files. Any `.md` file placed here becomes a page in the documentation.

Subdirectories create URL path segments. The directory structure maps directly to the output structure.

Theme Directory

The `theme/` directory holds customization files:

- `custom.css` — your CSS overrides, loaded after the default theme
- `templates/layout.html` — optional full template override using Tera

Output Directory

The `dist/` directory (configurable via `output_dir`) is generated by `docanvil build`. It contains the complete static site ready for deployment, including a `search-index.json` file when search is enabled. Use `--clean` to remove it before rebuilding.

i18n Project Layout

When localisation is enabled, content files use locale suffixes and the build output is split into per-locale directories:

```
my-project/
  docanvil.toml           # Includes [locale] section
  nav.toml                # Default navigation (fallback)
  nav.fr.toml             # French-specific navigation (optional)
  docs/
    index.en.md           # English home page
    index.fr.md           # French home page
    guides/
      getting-started.en.md
      getting-started.fr.md
  dist/                   # Build output
    js/docanvil.js         # Shared across locales
    robots.txt
    sitemap.xml            # Contains all locales
    404.html
    en/
      index.html
      guides/
        getting-started.html
      search-index.json    # English search index
    fr/
      index.html
      guides/
        getting-started.html
      search-index.json    # French search index
```

See Localisation for a complete guide to setting up multi-language docs.

Page Discovery

DocAnvil discovers pages by walking the content directory recursively and collecting all `.md` files. Each file becomes a page with a slug, title, and output path.

Slug Derivation

The slug is the file path relative to the content directory, with the `.md` extension removed and backslashes normalized to forward slashes:

SOURCE FILE	SLUG	OUTPUT FILE	TITLE
`docs/index.md`	`index`	`index.html`	Home
`docs/guides/getting-started.md`	`guides/getting-started`	`guides/getting-started.html`	Getting Started
`docs/reference/cli.md`	`reference/cli`	`reference/cli.html`	Cli
`docs/writing/wiki-links.md`	`writing/wiki-links`	`writing/wiki-links.html`	Wiki Links

Front Matter Slug Overrides

When a page has a `title` in its front matter, the slug is derived from the title instead of the filename. An explicit `slug` front matter field takes priority over both. This lets you use organizational prefixes in filenames while keeping clean URLs:

SOURCE FILE	FRONT MATTER	SLUG	OUTPUT FILE
`docs/01-intro.md`	`{"title": "Introduction"}`	`introduction`	`introduction.html`
`docs/guides/01-setup.md`	`{"title": "Setup Guide"}`	`guides/setup-guide`	`guides/setup-guide.html`
`docs/faq-page.md`	`{"slug": "faq"}`	`faq`	`faq.html`

Directory prefixes are preserved — only the filename portion changes. Pages named `index.md` are exempt from title-derived slugs. See Front Matter for full details.

Wiki-links using the old filename-based slug continue to resolve after a slug override.

Locale-Aware Slugs

When i18n is enabled, the locale suffix is stripped before building the slug. The locale is tracked separately and the output path includes a locale prefix:

SOURCE FILE	SLUG	LOCALE	OUTPUT FILE
`docs/index.en.md`	`index`	`en`	`en/index.html`
`docs/index.fr.md`	`index`	`fr`	`fr/index.html`
`docs/guides/setup.en.md`	`guides/setup`	`en`	`en/guides/setup.html`
`docs/guide.md`	`guide`	*(default)*	`en/guide.html`

Files without a locale suffix are assigned the default locale. See Localisation for details.

Title Generation

Titles are derived from the slug's last path component:

- `index` becomes "Home"
- Hyphens and underscores become spaces
- Each word is capitalized (`getting-started` → "Getting Started")

Titles appear in the sidebar navigation (unless overridden by `label` in `nav.toml`) and in the page's `<title>` tag.

Auto-Discovery Navigation

When `nav.toml` is absent, the navigation tree is built from the directory structure:

- Top-level `.md` files become root nav items
- Subdirectories become collapsible groups, labeled with the directory name in title case
- Files within directories become children of their group
- Everything is sorted alphabetically

> Note
>
> Files are sorted by path during discovery, ensuring deterministic navigation order regardless of filesystem ordering.

Related Pages

- Configuration — `docanvil.toml` and `nav.toml` options
- Theming — CSS variables, custom CSS, and template overrides
- Installation — creating your first project

Roadmap

DocAnvil follows Semantic Versioning. Here's where things stand and where they're headed.

Current version: 1.0.0

1.1.x — Core Feature Enhancements

✅ Localisation support (multi-language docs, per-locale nav and search, language switcher) ✅ PDF export (`docanvil export pdf` — Chrome-based, cover pages, RTL, per-locale, custom paper size) ⬜ Doc versioning support ⬜ Style linting

1.2.x — Extensibility

WASM plugin system (v1), additional CLI flags, more diagnostics, and template enhancements. Fully backward compatible.

1.3.x+ — Ecosystem Growth

Plugin hooks, performance optimizations, incremental builds, caching, and a plugin SDK crate. Breaking changes require a major release.

2.0 and Beyond

A 2.0 release would only happen if there's a clear need — configuration redesign, plugin API v2, fundamental output model changes, or major architectural shifts. It should be deliberate and well-justified.